BRANCH DRE/PAYPAL-ALLOWANCES · PAYPAL SANDBOX ONLY

PayPal allowances: what to test

A buyer gives an agent a spending allowance backed by a saved PayPal payment method, the agent buys from a small seeded marketplace inside it, and the merchant answers a PayPal dispute with the stored record. Nothing here changes the Colosseum relay, the Solana MCP tools or the website.

Watch first: **paypal-allowances-demo.mp4** (2:08) in the demo folder shows the whole cycle in the real workspace UI with PayPal and the API simulated. It is not payment evidence. The steps in this guide use the real backend and the PayPal sandbox.

Three things are new

Allowances	A buyer sets a name, total cap, per-purchase cap, merchants, an optional agent, purchases per day, a cooldown and an expiry, then saves a PayPal method for it (a vault token that never leaves the server). Two modes: **Agent pays** charges at purchase time; **Ask me first** files a request the buyer approves or denies from an inbox, and charges on approval. Every quote and purchase runs the same nine rules in the same order (status, expiry, agent, merchant, per-purchase cap, total cap, max per day, cooldown, vault); the first failing rule names the `reason_code`.
Marketplace	Four seeded merchants with fixed prices in cents. A basket is quoted against an allowance before anything is bought. A captured purchase is a task receipt with one unit per item: the merchant fulfils or fails each one and the agent can ask for a refund of exactly the failed ones.
Dispute evidence	The merchant syncs open PayPal disputes on captured purchases, previews an evidence pack built only from the stored record (rule checks, who approved, delivery per unit, refunds, digests) and submits it once. PayPal decides the outcome.

How it sits beside the document service

The existing PayPal service (quote a document job, pay by checkout, deliver, accept or challenge, refund) is untouched. Allowances reuse its pieces: the same access codes and roles, the same task record and receipt page, the same per-unit fulfilment and refund path, the same webhook inbox and share link. A purchase is a task whose receipt carries a `purchase` block and whose units have a price instead of a document hash. The new write routes sit behind one flag, `CHAINPAY_PAYPAL_ALLOWANCES`; with it off the service behaves as before.

The cycle at a glance

1. Set caps	Buyer builds an allowance and saves a PayPal method for it. Nothing charged.
2. Quote	Agent prices a basket against the allowance. The verdict lists all nine rules.
3. Buy or ask	Agent pays (charged now) or requests approval (buyer decides in the inbox).
4. Fulfil	Merchant marks each unit delivered or failed. The record keeps both.
5. Refund	Agent requests a refund for the failed units; a merchant approves it.
6. Dispute	Merchant syncs PayPal disputes and submits the evidence pack once.
7. Share	Buyer shares a redacted receipt that opens without signing in.

SETUP

Run it locally

You need Postgres, Rust, Node 20+, a PayPal sandbox REST app (client id and secret) and an OpenRouter key with a spend cap. Full notes are in **deploy/paypal/README.md**; `deploy/paypal/.env.example` is the checklist for a fresh environment. Rust reads exported variables, not the file. Never reuse the relay's environment or database.

Backend

```
cargo run -p chainpay-backend --features paypal --bin chainpay-paypal
```

`CHAINPAY_PAYPAL_ENABLED`	`true`. Off, the service answers 503 on /healthz and has no payment routes.
`CHAINPAY_PAYPAL_NEW_PURCHASES`	`true` for the test. Off, checkout and allowance purchases are refused; reads and recovery still work.
`CHAINPAY_PAYPAL_ALLOWANCES`	`true`. Gates `POST /allowances` and `POST /purchases`. Reads, approvals, fulfilment and dispute routes stay available when it is off.
`CHAINPAY_PAYPAL_DATABASE_URL`	A separate database whose name starts `chainpay_paypal_`. `DATABASE_URL` is never a fallback.
`CHAINPAY_PAYPAL_PUBLIC_ORIGIN`	The workspace origin, `http://localhost:5197` for the Vite dev server. PayPal returns the buyer here after approving the vault.
`CHAINPAY_PAYPAL_HOST`, `PORT`	`127.0.0.1` and `8081`; the frontend expects the API on 8081.
`PAYPAL_API_BASE`	`https://api-m.sandbox.paypal.com`. Only the sandbox origin is accepted.
`PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`	The sandbox REST app. The app needs the **Vault** feature for PayPal wallet vaulting and the **Disputes** feature for sync.
`PAYPAL_WEBHOOK_ID`	The id of the webhook registered on that app (next section).
`CHAINPAY_PAYPAL_AI_KEY`	A dedicated OpenRouter key. The allowance path never calls it, but startup requires it.
`CHAINPAY_PAYPAL_SUBSCRIPTIONS`	`false` for this test.

Webhooks to register on the sandbox app

Point the webhook at `/v1/paypal/webhooks` (use a tunnel such as ngrok for a local run) and subscribe to the order and capture events the document service already uses (`CHECKOUT.ORDER.APPROVED`, `CHECKOUT.ORDER.COMPLETED`, `PAYMENT.CAPTURE.COMPLETED`, `PAYMENT.CAPTURE.DENIED`, `PAYMENT.CAPTURE.REVERSED`, `PAYMENT.CAPTURE.REFUNDED`, `PAYMENT.REFUND.PENDING`, `PAYMENT.REFUND.COMPLETED`) plus the six this branch adds:

```
VAULT.PAYMENT-TOKEN.CREATED   VAULT.PAYMENT-TOKEN.DELETED   VAULT.PAYMENT-TOKEN.DELETION-INITIATED
CUSTOMER.DISPUTE.CREATED      CUSTOMER.DISPUTE.UPDATED      CUSTOMER.DISPUTE.RESOLVED
```

A deleted vault token puts the allowance's payment method back to not ready, so the next quote fails on the `vault` rule. A dispute event only marks a capture the service already knows; everything else stays in the inbox.

Access codes

Issue three grants in one workspace from a shell that has the backend variables loaded. The agent shares the buyer's principal; the merchant is its own. Each prints a 64-character code: treat them like passwords and keep them out of logs and recordings.

```
cargo run -p chainpay-backend --features paypal --bin chainpay-paypal -- issue-access test-ws test-buyer buyer
cargo run -p chainpay-backend --features paypal --bin chainpay-paypal -- issue-access test-ws test-buyer agent
cargo run -p chainpay-backend --features paypal --bin chainpay-paypal -- issue-access test-ws test-merchant merchant
```

Frontend

```
cd frontend
VITE_CHAINPAY_PAYPAL=true VITE_CHAINPAY_PAYPAL_API_ORIGIN=http://127.0.0.1:8081 npm run dev:paypal
```

Open `http://127.0.0.1:5197/paypal/`. Use a separate browser profile or private window per role; signing out clears only the current tab.

SETUP

Check the sandbox app first

Two PayPal features have to be switched on for the app before the walkthrough works, and the error you get later is not obvious. This script asks PayPal directly. It uses the secret for the token call only and never prints it.

```
PAYPAL_CLIENT_ID=... PAYPAL_CLIENT_SECRET=... node scripts/paypal-sandbox-check.mjs
```

What each line means

`token: ok`	The client id and secret are a valid sandbox app. Anything else here means the credentials are wrong, not the features.
`vault setup token (paypal wallet): HTTP 201 status=PAYER_ACTION_REQUIRED approve link: https://www.sandbox.paypal.com/...`	The app can vault a PayPal wallet. This is the link the buyer is sent to in walkthrough step 2.
`vault setup token (paypal wallet): HTTP 4xx error: NOT_AUTHORIZED ...`	The Vault feature is not enabled on the app (Apps & Credentials → the app → Features → Vault). Either enable it or choose **Sandbox card** as the payment method when building the allowance; the test card `4111111111111111` with any future expiry works there.
`disputes api: HTTP 200 enabled`	The merchant's **Sync from PayPal** will work.
`disputes api: HTTP 4xx not enabled (...)`	The Disputes feature is off on the app. Sync answers with PayPal's error and the disputes table stays empty; everything before step 7 still works.

Health check: `GET http://127.0.0.1:8081/healthz` answers `{"status":"ready"}` once enabled and the database answers. "disabled" means `CHAINPAY_PAYPAL_ENABLED` is not exactly `true`; "unavailable" means the database is unreachable. A ready health response proves configuration, not PayPal acceptance.

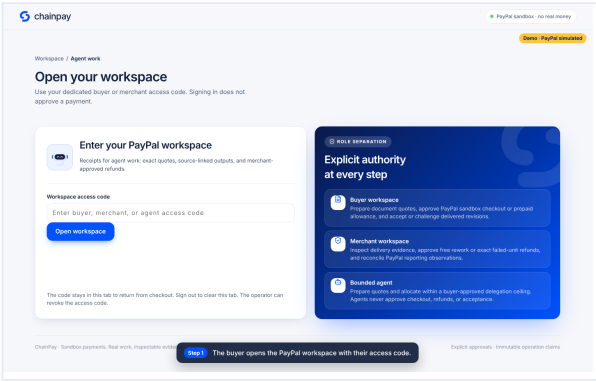
What the walkthrough uses

Two allowances. You create **Weekend errands** in step 2 ($120 total, $40 per purchase, Corner Grocer and Riverside Box Office, Agent pays, PayPal sandbox). Before step 4, create a second one as the buyer named **Show tickets**: Ask me first, Riverside Box Office only, $300 total, $100 per purchase, agent `test-buyer`, and complete its PayPal approval too. The demo recording seeds this one; your run makes it by hand.

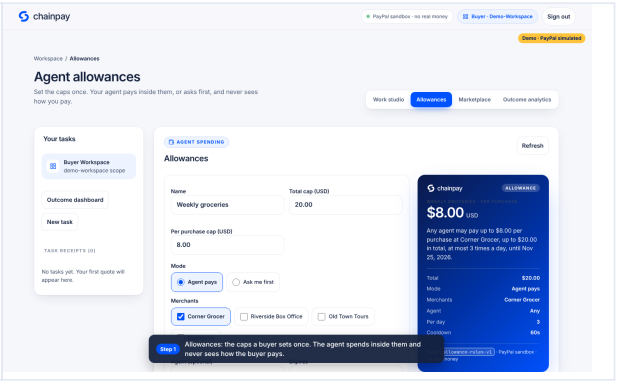
WALKTHROUGH

Step 1 · Sign in and open Allowances (buyer)

#	Do this	You should see
1	Open the workspace and paste the **buyer** access code. Click **Open workspace**.	The view switcher: Work studio, Allowances, Marketplace, Outcome analytics. The header says **Buyer · test-ws**.
2	Click **Allowances**.	"Agent allowances" with the **New allowance** form on the left and a blue allowance card on the right. Below it, **Waiting for you (0)** and no rows yet.



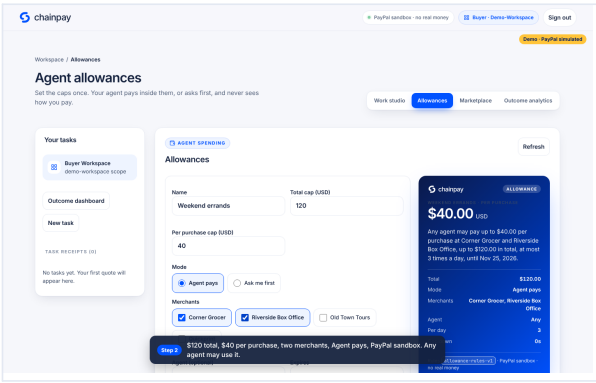
shots/01-sign-in.png. The access-code sign-in.



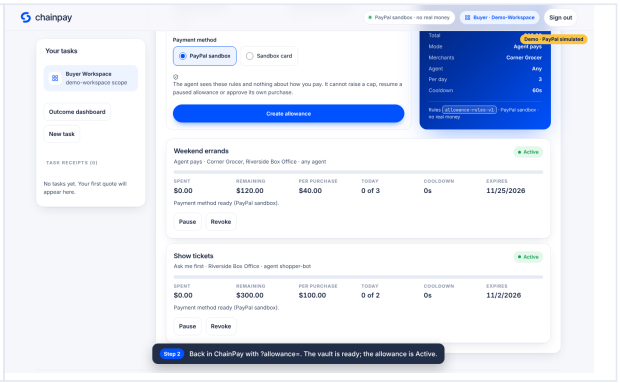
shots/02-allowances.png. The empty Allowances view for a buyer.

Step 2 · Build "Weekend errands" and approve it on PayPal (buyer)

#	Do this	You should see
1	Name **Weekend errands**, Total cap **120**, Per purchase cap **40**, Mode **Agent pays**, tick **Corner Grocer** and **Riverside Box Office**, leave Agent empty, Cooldown **0**, Max purchases per day **3**, Payment method **PayPal sandbox**.	The card reads "Any agent may pay up to $40.00 per purchase at Corner Grocer and Riverside Box Office, up to $120.00 in total, at most 3 times a day, until <date>."
2	Click **Create allowance**.	A new row **Weekend errands** with status **Active**, "Payment method not ready" and a link **Continue PayPal approval**.
3	Click **Continue PayPal approval**. Log in with the sandbox **personal** account and agree to save the payment method.	PayPal's own approval page for a saved payment method. Nothing is charged.
4	You are returned to the workspace with `?allowance=` in the URL.	The row now says **Payment method ready (PayPal sandbox)**. Spent $0.00, Remaining $120.00.



shots/03-builder-filled.png. The filled form and the card it produces.

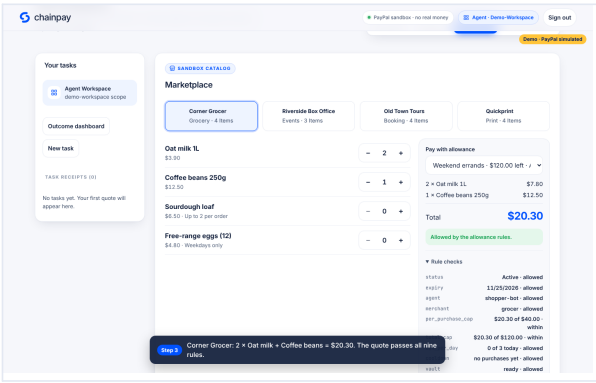


shots/06-allowance-ready.png. Back from PayPal: the vault is ready and the allowance is Active.

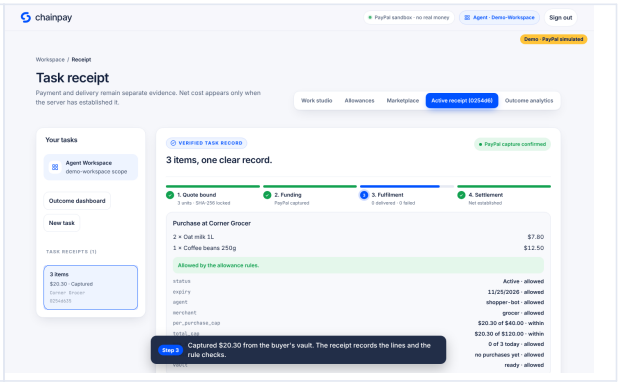
If PayPal shows an error instead of the approval page, the app does not have the Vault feature. Revoke this allowance and create it again with **Sandbox card** as the payment method; the rest of the walkthrough is the same.

Step 3 · The agent buys groceries (agent)

#	Do this	You should see
1	Sign out. Paste the **agent** code and open the workspace. Click **Allowances**.	The agent sees **Weekend errands** with its caps and merchants, but no payment method, no vault status and no Pause or Revoke buttons.
2	Click **Marketplace**, pick **Corner Grocer**, add Oat milk 1L ×2 and Coffee beans 250g ×1. Pay with allowance: **Weekend errands**.	Total **$20.30**. The quote pane says **Allowed by the allowance rules.** Open **Rule checks**: nine rows, every one "allowed" or "within"; the expiry row shows a date, the cooldown row "no purchases yet".
3	Click **Buy now · $20.30**.	The receipt opens with the status **PayPal capture confirmed**; the sidebar shows **3 items**, **$20.30 · Captured**, Corner Grocer. The purchase block lists the lines and the rule checks; each unit shows its price and is **Pending**.



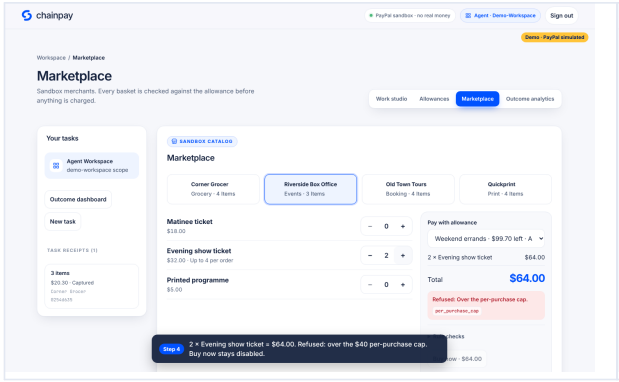
shots/09-market-quote-allowed.png. The grocery quote passes all nine rules.



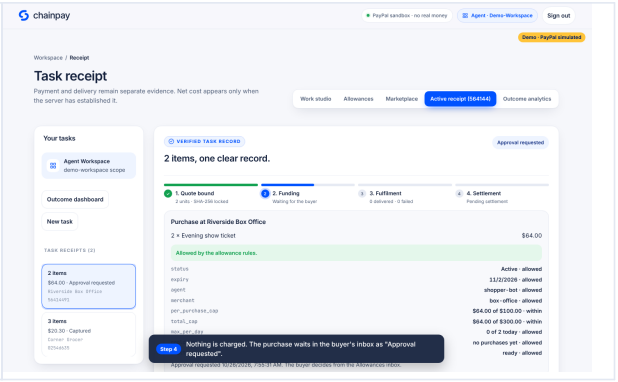
shots/10-receipt-captured.png. The receipt right after capture. Units carry a price, not a document hash.

Step 4 · Over the cap, then an ask-first request (agent)

#	Do this	You should see
1	Still in Marketplace: pick **Riverside Box Office**, add Evening show ticket ×2, keep **Weekend errands** selected.	Total $64.00. The verdict is **Refused: Over the per-purchase cap.** with the code `per_purchase_cap`. **Buy now** is disabled. In Rule checks, per_purchase_cap reads "$64.00 of $40.00 · over".
2	Change the allowance to **Show tickets**.	**Allowed by the allowance rules.** The button is now **Buy now · $64.00**.
3	Click it.	A receipt with the status **Approval requested**. The sidebar shows **2 items**, **$64.00 · Approval requested**. Nothing was charged: no capture id, Funding reads "Waiting for the buyer".



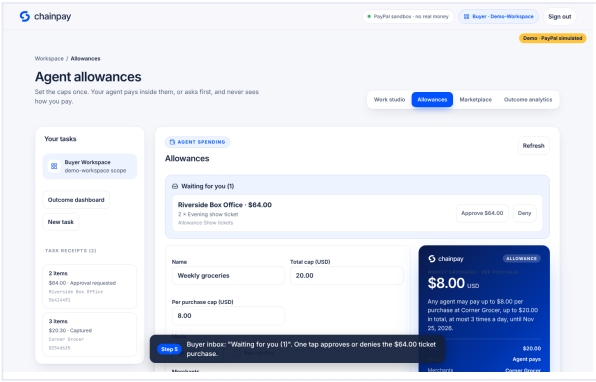
shots/11-market-over-cap.png. Refused on the per-purchase cap; the button stays off.



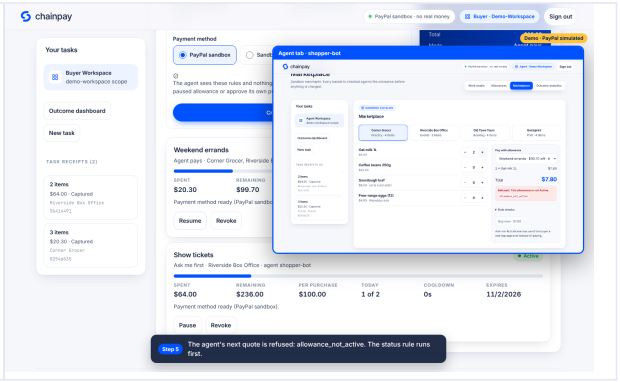
shots/13-receipt-approval-requested.png. An ask-first purchase waits for the buyer. No capture exists yet.

Step 5 · Approve, then pause (buyer)

#	Do this	You should see
1	Sign out, sign in as the **buyer**, click **Allowances**.	**Waiting for you (1)** with the $64.00 request from Riverside Box Office and two buttons, **Approve $64.00** and **Deny**.
2	Click **Approve $64.00**.	The receipt opens with **PayPal capture confirmed**. The purchase block says "Approved by test-buyer <time>" and the rule checks were run again at approval time. Show tickets now shows Spent $64.00, Today 1 of 2.
3	In a second window signed in as the agent, open Marketplace, pick Corner Grocer, add Oat milk ×1 under Weekend errands.	**Allowed by the allowance rules.**
4	Back as the buyer, on the **Weekend errands** row click **Pause**.	The pill reads **Paused**; the buttons become **Resume** and **Revoke**.
5	In the agent window change the basket (add one more Oat milk).	**Refused: This allowance is not Active.** with the code `allowance_not_active`. The status rule is the first row. Refresh the agent's Allowances view: Weekend errands is no longer listed.
6	Optional: as the buyer **Resume** it, then test a denial. Have the agent request $18.00 (Matinee ×1) under Show tickets and click **Deny** with a reason.	The receipt says "Denied by the buyer. Nothing was charged." and the rule checks gain a `buyer_decision` row. If an allowance is paused between request and approval, approving it instead gives "Denied by the allowance rules at approval time" with `approved_by` = `rules`.



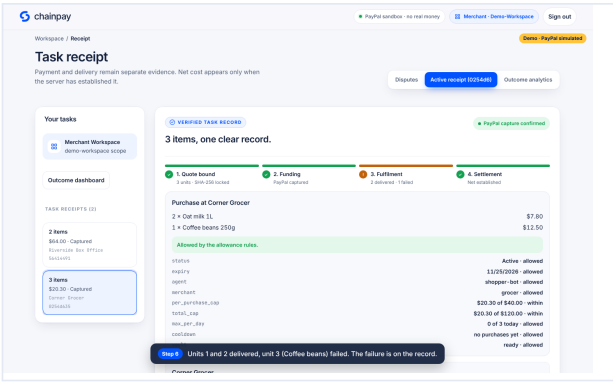
shots/14-inbox.png. The buyer's inbox: one request, approve or deny.



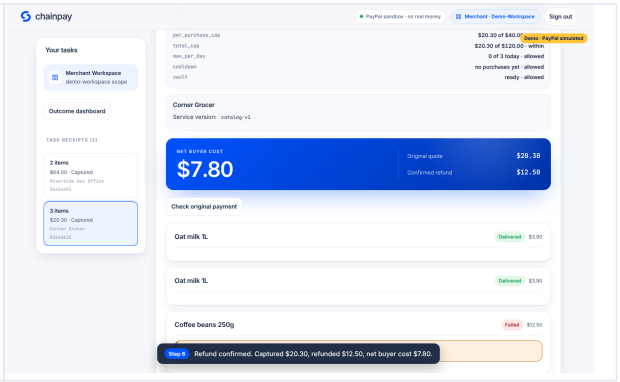
shots/18-agent-not-active.png. After the pause (buyer tab) the agent's next quote is refused (inset).

Step 6 · Fulfil, fail a unit, refund (merchant and agent)

#	Do this	You should see
1	Sign in as the **merchant**. Open the **$20.30** receipt from the sidebar.	Three units, each with **Fulfil unit N** and **Mark unit N failed** buttons. The merchant has no Allowances or Marketplace view.
2	Click **Fulfil unit 1**, **Fulfil unit 2**, **Mark unit 3 failed**.	The timeline reads **2 delivered · 1 failed**. Unit 3 (Coffee beans 250g, $12.50) shows **Failed**.
3	Sign in as the **agent**, open the same receipt, click **Request refund for failed units**.	**Refund requested: $12.50 for unit 3.** A merchant must approve it. The agent has no approve button.
4	Sign in as the **merchant**, open the receipt, click **Approve $12.50 refund**.	Within a few seconds the status is **Refund confirmed**. Net buyer cost **$7.80**: original quote $20.30, confirmed refund $12.50.



shots/20-merchant-fulfilled.png. Delivered and failed units on one record.



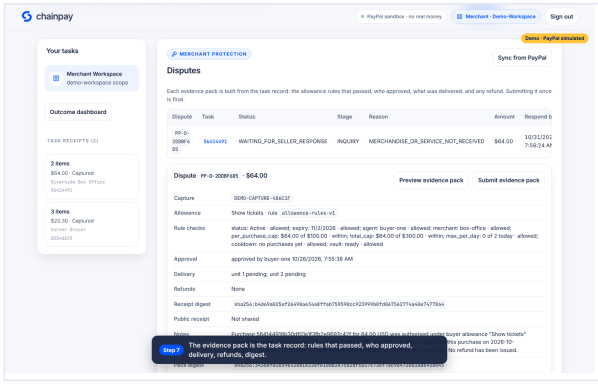
shots/24-refund-confirmed.png. After the refund: captured $20.30, refunded $12.50, net $7.80.

Step 7 · A dispute on the ticket capture (merchant)

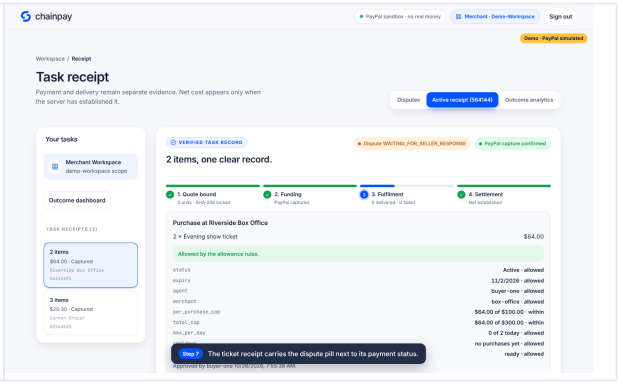
Raise the dispute at **sandbox.paypal.com** as the personal account that paid: Activity → the $64.00 payment → Report a problem → Item not received. Then, as the merchant:

#	Do this	You should see
1	Click **Disputes**, then **Sync from PayPal**.	One row: the dispute id, the task id, status **WAITING_FOR_SELLER_RESPONSE**, stage **INQUIRY**, reason `MERCHANDISE_OR_SERVICE_NOT_RECEIVED`, $64.00, evidence **Not sent**.
2	Click **Preview evidence pack**.	Capture id, allowance "Show tickets" with rule `allowance-rules-v1`, all nine rule checks, "approved by test-buyer <time>", delivery per unit, refunds, receipt digest, notes and the pack digest.
3	Click **Submit evidence pack**.	**Evidence submitted**; the table's evidence column reads **Submitted**. Clicking it again is refused: one submission per dispute.

#	Do this	You should see
4	Click the task id in the table.	The $64.00 receipt with a **Dispute WAITING_FOR_SELLER_RESPONSE** pill next to its payment status.



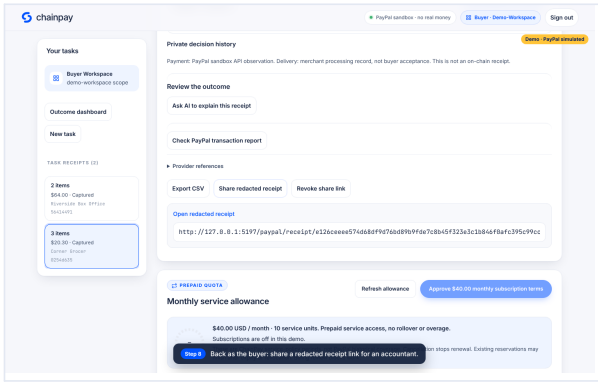
shots/27-evidence-preview.png. The evidence pack is built from the stored record only.



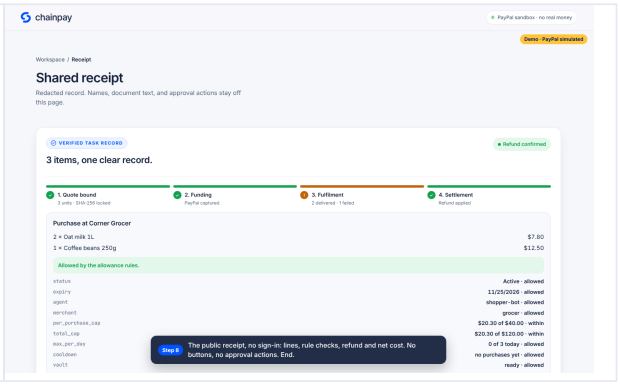
shots/29-receipt-dispute-pill.png. The disputed receipt carries the pill until PayPal resolves the case.

Step 8 · Share a redacted receipt (buyer)

#	Do this	You should see
1	Sign in as the **buyer**, open the **$20.30** receipt, click **Share redacted receipt**.	A **Public receipt URL** field with a long token.
2	Open that URL in a private window.	"Shared receipt" with the status **Refund confirmed**, the lines, the nine rule checks, the refund and net $7.80. No names, no approval, fulfil or refund buttons, no quote digest.
3	Back in the workspace click **Revoke share link**, reload the private window.	The link no longer opens.



shots/30-share-link.png. The share link, issued by the buyer.



shots/31-public-receipt.png. The public receipt, opened without signing in.

AGENT TOOLS

The seven allowance tools

The stdio MCP entrypoint (`node mcp-server/dist/paypal.js`, see **docs/guides/paypal-agent.md**) gains seven tools when the API has `CHAINPAY_PAYPAL_ALLOWANCES=true`. It needs an **agent** grant in `CHAINPAY_PAYPAL_AGENT_TOKEN`; a buyer or merchant grant is refused at the MCP boundary. There is still no tool that approves, captures or refunds money.

Tool and arguments	What it does
`paypal_find_allowance` **no arguments**	The Active allowances this agent may spend from: caps, remaining cents, merchants, mode. The vault block never appears. A paused, revoked or other-agent allowance is simply absent.
`paypal_search_catalog` `q` **(text, optional)**	Merchants and items with integer `price_cents` and availability (`always`, `weekdays`, `max_per_order:N`).
`paypal_quote_purchase` `merchant`, `items` **[{id, qty}]**, `allowance_id` **(optional)**	Lines, `amount_cents` and, when an allowance is named, the policy verdict with all checks and a `reason_code` when refused. No task, no PayPal call.
`paypal_purchase` `request_id` **(16–64 chars)**, `allowance_id`, `merchant`, `items`	Buys on an **agent_pays** allowance. The tool sends `mode: "agent_pays"`; the server refuses it on an ask_first allowance. Result `kind` is `receipt` or `refused` (with `reason_code`). The same request_id returns the same task.
`paypal_request_approval` **same arguments**	The same call with `mode: "ask_first"`. Creates the task with approval requested and returns `review_url`; refused on an agent_pays allowance, so it can never charge. Result `kind` is `approval_requested`.
`paypal_list_approvals` **no arguments**	This agent's requests still waiting on the buyer. A request the rules fail at approval time comes back `denied` with `approved_by: "rules"`.
`paypal_request_refund` `task_id` **(32 hex)**, `reason`	Records a refund request for the failed units with a reason. The merchant approves in the workspace.

A sample conversation

Run the entrypoint under an MCP client with the agent grant, then try this. The tool results below are shortened; amounts are exact.

User	What can you spend on groceries for me?
Agent	Calls `paypal_find_allowance`.
Result	`{"allowances":[{"id":"al-001","name":"Weekend errands","mode":"agent_pays","merchants":["grocer","box-office"],"per_purchase_cap_cents":4000,"total_cap_cents":12000,"spent_cents":0,"remaining_cents":12000,...}]}` (no vault block)
Agent	Weekend errands: up to $40.00 per purchase at Corner Grocer and Riverside Box Office, $120.00 left in total. Agent pays.
User	Get two oat milks and a bag of coffee beans from the grocer.

Agent	Calls `paypal_quote_purchase` with `merchant "grocer", items [{"id":"oat-milk","qty":2},{"id":"coffee-beans","qty":1}], allowance_id "al-001"`.
Result	`{"amount_cents":2030,"policy":{"allowed":true,"reason_code":null,"checks":[{"rule":"status","allowed":true,"observed":"Active"},...]}}`
Agent	Calls `paypal_purchase` with a fresh `request_id` and the same basket.
Result	`{"kind":"receipt","review_url":"http://127.0.0.1:5197/paypal/?task=...","receipt":{"captured":true,"gross":"20.30",...}}`
Agent	Done: $20.30 captured from your PayPal method. Receipt at the review_url.
User	Now two evening show tickets.
Agent	Calls `paypal_quote_purchase` for box-office under al-001.
Result	`{"amount_cents":6400,"policy":{"allowed":false,"reason_code":"per_purchase_cap",...}}`
Agent	Refused: $64.00 is over the $40.00 per-purchase cap on Weekend errands. Your Show tickets allowance is ask-first; shall I request approval there?
User	Yes.
Agent	Calls `paypal_request_approval` with `allowance_id "al-tickets"`.
Result	`{"kind":"approval_requested","review_url":"...?task=9379f5f3...","receipt":{"captured":false,"purchase":{"approval":"requested",...}}}`
Agent	Requested. Nothing is charged until you approve it in the Allowances inbox.

What should not happen: the agent is never shown a vault token; a `paypal_request_approval` on Weekend errands or a `paypal_purchase` on Show tickets is a 422 from the server; a retried purchase with the same request_id returns the first task, not a second charge.

REVIEW

What to check before merging

These are the properties the branch claims. Each one can be checked from the workspace or with curl against `http://127.0.0.1:8081/v1/paypal` using the access codes as bearer tokens.

Check	How	Pass when
A refused purchase cannot be checked out	Quote $64.00 under Weekend errands (step 4). Then POST `/purchases` with that basket anyway.	The UI keeps **Buy now** disabled. The API answers 200 with a task whose policy is `allowed:false`, `reason_code per_purchase_cap`, `captured:false` and no capture id. It is listed as **Refused** and the merchant gets 422 on `/fulfil`.
request_id replay	POST `/purchases` twice with the same `request_id` and the same body; then once more with the same id and a different basket.	The second call returns the same task id, one capture, spent $20.30 not $40.60. The third call is 409.
The agent never sees the vault	GET `/allowances` and `/allowances/:id` with the agent code; grep the JSON.	No `vault` key, no `approval_url`, no token id. The buyer's view has `vault.status` and nothing more once it is ready.
Approve with a stale digest is 409	POST `/purchases/:id/approve` as the buyer with a `digest` that is not the receipt's `quote_digest`.	409 and the request stays **Approval requested**. With the right digest it captures.
Merchants get 403s	With the merchant code: POST `/allowances`, POST `/purchases`, POST `/purchases/:id/approve`. Also GET `/allowances`.	403 on all three writes. The GET answers `{"allowances":[]}`. The agent code gets 403 on `/allowances` POST, approve, deny and `/tasks/:id/refund`.
Evidence is submitted once	POST `/disputes/:id/evidence` with the preview's digest, then again.	First call: evidence **submitted**. Second: 409. A wrong digest is also 409 and nothing is sent.
Mode must match	POST `/purchases` with `mode:"ask_first"` on Weekend errands, and `mode:"agent_pays"` on Show tickets.	422 both times with a message naming the allowance's real mode. Nothing is created.
Pause is immediate	Pause Weekend errands while the agent has an allowed quote open, then let the agent buy.	The purchase is refused with `allowance_not_active`; no capture. Resume and it passes again. Revoke is final: `/status` back to Active is 409.
Re-check at approval	Request an ask-first purchase, pause the allowance, approve the request.	The receipt reads "Denied by the allowance rules at approval time", `approved_by:"rules"`, and the reserve is released (Spent unchanged).

Also run

```
cargo test -p chainpay-backend --features paypal --locked
cargo fmt --all -- --check
npm --prefix sdk test && npm --prefix mcp-server test
npm --prefix frontend run test:browser -- paypal-allowances paypal-disputes
```

The browser tests drive the same screens against `frontend/test/fixtures/paypal-allowances.tsx`, a mocked API in the page. They do not reach PayPal.

Known limits

Sandbox only	The service refuses any PayPal origin other than `api-m.sandbox.paypal.com`. Nothing here is a live payment.
Vault feature	Saving a PayPal wallet needs the Vault feature on the sandbox REST app. Without it the PayPal source fails at setup; the card source (test card `4111111111111111`) is the fallback and the rest of the flow is identical.
Disputes feature	Sync and evidence need the Disputes feature on the app. Without it **Sync from PayPal** shows PayPal's error and the table stays empty.
Weekdays are UTC	Items marked "Weekdays only" (eggs, bike rental, spiral binding) are checked against the UTC weekday, not the buyer's. On a Saturday evening in Bangkok they are still refused until 07:00 Monday local time.
Max per day is UTC too	The daily purchase count resets at 00:00 UTC.
Cooldown in the demo	The recording sets cooldown 0 so the steps run back to back. With a real value the second purchase under the same allowance is refused with `cooldown` until it passes.
Dispute outcome	ChainPay submits the pack once and records what PayPal says on the next sync or webhook. It does not decide, appeal or refund on a dispute.
Seeded catalog	Merchants and prices are fixed in `backend/src/paypal/catalog.rs`. There is no merchant onboarding; this is a test bed for the allowance rules.

If something breaks

Note the task id (first eight characters are in the sidebar) and the time. Do not create a new task to work around it: every purchase step can be recovered from the original task by its request_id, and that recovery is part of what is under test. Backend logs print to the terminal running `chainpay-paypal`.